



Disciplina: Programação 3 - Programação Funcional

Professor: Emanuel Barreiros

Assunto: (INTRO) Criando tipos e classes

Resumo: Utilize a linguagem de programação Haskell para resolver os problemas desta lista. Esta lista aborda o conceito de criação de tipos e classes.

- 1) Defina um tipo de dado chamado `DiaSemana` que represente os sete dias da semana (de `Segunda` a `Domingo`). Em seguida, implemente a função `ehFimDeSemana :: DiaSemana -> Bool` que retorna `True` se o dia fornecido for `Sabado` ou `Domingo`, e `False` para os demais dias da semana.
- 2) Defina um tipo de dado `Ponto2D` que represente um ponto no plano cartesiano através de duas coordenadas do tipo `Double`. Escreva uma função chamada `distanciaOrigem :: Ponto2D -> Double` que calcule a distância euclidiana desse ponto até a origem (0,0).
- 3) Defina um tipo de dado chamado `Cliente`. Um cliente pode ser uma `PessoaFisica` (que armazena o nome como `String` e a idade como `Int`) ou uma `PessoaJuridica` (que armazena a razão social como `String` e o ano de fundação como `Int`). Escreva uma função `obterNome :: Cliente -> String` que extraia e retorne apenas o nome/razão social do cliente, independentemente de seu tipo.
- 4) Crie sua própria estrutura de lista definindo um tipo `ListaInt`. Ela deve ter dois construtores: `Vazia` (representando o fim da lista) e `No` (que guarda um elemento do tipo `Int` e o restante da estrutura `ListaInt`). Em seguida, implemente de forma recursiva a função `somaLista :: ListaInt -> Int` para somar todos os inteiros contidos nessa lista.
- 5) Crie o operador `(>|) :: Int -> ListaInt -> ListaInt` que é capaz de inserir um elemento ao fim de uma `ListaInt`.
Exemplo:

```
ghci> 5 >| (No 1 (No 2 (No 2 (Vazia))))  
ghci> No 1 (No 2 (No 2 (No 5 Vazia)))
```
- 6) Crie o operador `(|<) :: Int -> ListaInt -> ListaInt` que é capaz de inserir um elemento no início de uma `ListaInt`.
Exemplo:

```
ghci> 5 |< (No 1 (No 2 (No 2 (Vazia))))
```



ghci> No 5 (No 1 (No 2 (No 2 Vazia)))

- 7) Crie um tipo de dado `CorSemaforo` com três construtores sem argumentos: `Verde`, `Amarelo` e `Vermelho`. Escreva uma função `proximaCor :: CorSemaforo -> CorSemaforo` que simule a transição automática de um semáforo de trânsito tradicional (onde `Verde` avança para `Amarelo`, `Amarelo` avança para `Vermelho`, e `Vermelho` retorna para `Verde`).
- 8) Crie um tipo de dado parametrizado `Opcional a` que possua dois construtores: `Nenhum` e `Dado a`. Escreva uma função `filtrarValores :: [Opcional a] -> [a]` que receba uma lista nativa do Haskell composta por elementos do tipo `Opcional a` e retorne uma lista contendo apenas os valores desempacotados que estavam dentro do construtor `Dado`, descartando todas as ocorrências de `Nenhum`.